

Apache Spark, Processamento em Lote e em Tempo Real

Turma: 2º Ano A/B – Integral Prof.: Jorge Frias Data: ____ / 09 / 2026

No Bloco 1 você viu a arquitetura do Big Data (coleta, armazenamento, processamento) e os 3 Vs. Agora, a etapa de **processamento**: depois de coletado e guardado, como o dado vira informação útil? Duas analogias: **juntar a roupa da semana e lavar tudo de uma vez** (lote) × **a fritadeira elétrica, que vai fritando conforme você coloca** (tempo real).

Parte 1 Lote e tempo real

Processamento em lote (batch)

Os dados são acumulados durante um período e processados todos juntos, em blocos, num horário programado — muitas vezes de madrugada.

- **Latência alta** (minutos a horas) — e tudo bem, a resposta não precisa ser imediata.
- **Vazão alta**, previsível e mais barato.

Ex.: folha de pagamento mensal, relatório de vendas do mês, faturas de cartão.

Ferramentas: Hadoop MapReduce, Spark, Apache Hive.

Processamento em tempo real (streaming)

Cada dado é processado assim que chega — registro a registro, ou em micro-lotes de poucos segundos. Fluxo contínuo, o sistema sempre ligado.

- **Latência baixa** (segundos ou menos).
- Mais **complexo e caro**: infraestrutura pronta pra picos.

Ex.: fraude no cartão na hora da compra, trânsito no GPS, recomendação de vídeo ao vivo, alerta de sensor.

Ferramentas: Kafka, Flink, Storm, Spark Structured Streaming.

TABELA COMPARATIVA

ASPECTO	LOTE (BATCH)	TEMPO REAL (STREAMING)
Quando é processado	tudo de uma vez, agendado	na hora em que cada dado chega
Latência da resposta	alta (minutos a horas)	baixa (segundos ou menos)
Volume por execução	muito grande de uma vez	pequeno e constante
Custo e complexidade	menor	maior
Exemplo	folha de pagamento mensal	fraude no cartão na hora da compra

COMO DECIDIR ENTRE LOTE E TEMPO REAL

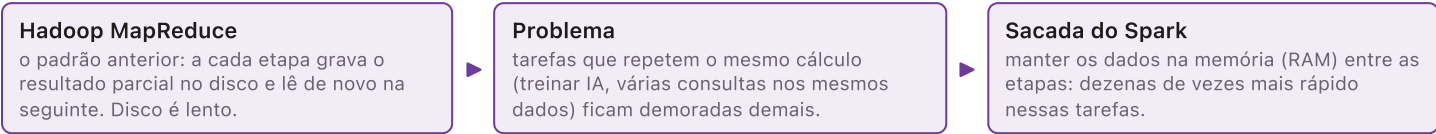
- 1 A decisão precisa ser tomada **no instante em que o dado acontece** (ex.: bloquear uma compra suspeita agora)? → **tempo real (streaming)**
- 2 Dá pra **esperar juntar um volume grande** e rodar de tempos em tempos (uma vez por hora, por dia, por mês)? → **lote (batch)**
- 3 Precisa dos dois? **Arquitetura híbrida**: tempo real pro que é urgente (alerta na hora) + lote pro que é volumoso e pode esperar (relatório completo). É o que a maioria das empresas grandes faz.

Parte 2 A Apache Software Foundation e o Spark

A **Apache Software Foundation** (ASF, criada em 1999, EUA) é uma organização sem fins lucrativos que mantém centenas de projetos **open source** (código aberto), com governança por mérito e a permissiva Licença Apache 2.0. "Apache" no nome (Spark, Kafka, Hadoop, Flink) = projeto mantido pela comunidade, que não pertence a uma empresa só e não some se ela mudar de ideia.

O **Apache Spark** é um **motor de processamento distribuído** para dados em larga escala: divide o trabalho entre vários computadores (um *cluster*) que trabalham em paralelo, como se fossem um só. Nasceu em Berkeley (2009) e foi doado à Apache em 2013.

POR QUE O SPARK SURTIU



Módulos: Spark Core (motor e memória), Spark SQL / DataFrames, Structured Streaming (tempo real), MLlib (machine learning), GraphX (grafos). **Linguagens:** Python (PySpark, a mais usada em Ciência de Dados), Scala, Java, R, SQL.

PRÓS E CONTRAS DO SPARK

Prós

- **Rápido** — processamento em memória.
- **Unificado** — lote, tempo real, SQL e ML no mesmo motor, com a mesma forma de programar.
- **Várias linguagens** e comunidade grande; roda nas principais nuvens.

Contras

- **Consome muita RAM** — memória é cara.
- **Exagero pra dados pequenos** — se cabem num PC só, ele só adiciona complexidade.
- **Curva de aprendizado;** e o streaming é em micro-lotes (o Flink tem latência menor).

CONCORRENTES

FERRAMENTA	FORTE EM	RESUMO
Hadoop MapReduce	lote	Padrão antigo. Maduro e barato, mas lento (usa disco) e trabalhoso de programar.
Apache Flink	tempo real	Streaming de verdade, evento a evento, latência mínima. Também faz lote.
Apache Storm	tempo real	Pioneiro de streaming, baixa latência; hoje pouco usado.
Dask	Python, escala menor	Paraleliza pandas e numpy. Mais simples que o Spark, bom pra dados médios; não escala tão longe.

O MESMO CÓDIGO FAZ OS DOIS · PIPELINE

O **Structured Streaming** trata o fluxo contínuo como **uma tabela que nunca para de crescer** — o mesmo código serve pro relatório mensal (lote) e pro painel ao vivo (streaming). Pipeline típico: **Fonte** (arquivos, banco, HDFS, S3, Kafka) ► **Leitura** (distribui as partições no cluster) ► **Transformações** (filtrar, juntar, agregar) ► **Ação** (gravar, contar, mostrar — dispara o processamento) ► **Destino** (arquivo, tabela, painel, alerta). O Spark só executa ao chegar numa **ação**; antes disso é só um plano que ele otimiza — *avaliação preguiçosa* (lazy evaluation).

Fim Exercícios

Responda no documento indicado, com suas próprias palavras. Nas classificações, o que vale ponto é a justificativa.

- 1
- Diferencie **processamento em lote** e **tempo real**, com um exemplo **novo** de cada.
- 2
- Classifique como **lote** ou **tempo real** e justifique: **a)** relatório de vendas do mês; **b)** fraude no cartão na hora da compra; **c)** rota no GPS enquanto o motorista dirige; **d)** juros mensais de todas as contas do banco; **e)** recomendação de vídeo enquanto o usuário assiste; **f)** fechamento da folha de pagamento.
- 3
- Por que o Apache Spark costuma ser mais rápido que o Hadoop MapReduce? O que ele faz de diferente? (dica: disco x memória)
- 4
- Escolha a ferramenta (**Spark, Flink, MapReduce ou Dask**) e justifique: **a)** startup pequena, equipe só de Python, ~10 GB por análise; **b)** banco que precisa reagir a fraude em milissegundos; **c)** um processamento gigante por dia, de madrugada, com pouco orçamento; **d)** time que quer um motor só pra lote, streaming e machine learning.